Re-Enforcement Learning.

Dr. Kaneeka Vidanage

Overview

- Reinforcement learning (RL) is like teaching a dog to perform tricks by rewarding good behavior.
    - **Agent**: Think of it like a dog you're training.
    - **Environment**: This is the world around the agent, like your home or a park where the dog is learning.
    - **Actions**: These are the things the dog can do, like sit, roll over, or fetch.
    - **Rewards**: These are treats you give the dog when it does something right.

Reflection.

1. **Learning by Trying**: The dog (agent) tries different actions to see what happens.
2. **Getting Feedback**: When the dog does something you like, like sitting when asked, you give it a treat (reward). If it does something wrong, like chewing your shoes, you don't reward it (or might scold it).
3. **Learning What's Best**: Over time, the dog learns which actions get the most treats and starts doing those more often.

In Computer Terms:

- **Agent**: A computer program or model.
- **Environment**: The system or scenario where the agent operates, like a game or a self-driving car simulation.
- **Actions**: Choices the agent can make, like moving left or right in a game.
- **Rewards**: Points or feedback the agent gets based on its actions, helping it learn the best strategies.

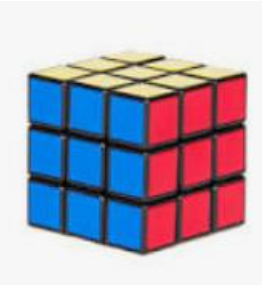
Reinforcement learning is all about trial and error, where the agent learns the best way to behave in a given situation by trying different actions and learning from the outcomes (rewards or penalties).

Popular Re-enforcement Algorithms.

- Q- Learning.
- Deep Q-Networks (DQN).
- Proximal Policy Optimization (PPO)
- Policy Gradient Methods.
- A3C (Asynchronous Advantage Actor-Critic)

Deep Q-Networks (DQN).

- Widely used.
- Improved version of Q- Learning.
- Combination of Q Learning + Deep Learning.



- Q Learning : -
  - Imagine you're learning to play a simple game, and you keep a list (a table) of actions in each situation (state) to get the highest score (reward).
  - But if the game is becoming complex with lots of different situations, the list would be too long and hard to manage.
- DQN :-
  - Instead of keeping a giant list, you use a neural network to guess the best action to take in any situation.

DQN in Operation.

- Imagine you're teaching a computer to play an old-school video game, like "Space Invaders“ : -
  - **State**: The current screen of the game (what the game looks like at this moment).
  - **Actions**: The possible moves the computer can make, like moving left, right, or shooting.
  - **Rewards**: Points the computer earns for hitting an enemy.
- DQN was famously used by Google DeepMind to create an AI that could play and master Atari games, sometimes even outperforming human players.
- DQN could learn complex strategies and adapt to different game scenarios, making it a powerful tool for various applications like robotics, self-driving cars, and more.

How DQN learns to play Space Invaders.

- **Observation**: The computer sees the game screen and processes it as an image.
- **Prediction**: The DQN uses a neural network to predict the best move (action) based on the current game screen (state).
- **Action**: The computer makes that move in the game.
- **Feedback**: The game gives feedback (reward or punishment). If the computer hits an enemy, it gets points (positive reward). If it loses a life, it loses points (negative reward).
- **Learning**: The DQN updates its neural network based on the feedback, learning which actions are best over time.

DQN for Self Driving Car.

- Objective → Train the car (agent) to find the best path to the destination using a DQN.
- Environment →
  - **Grid / Region**: A 5x5 grid representing a simplified map car can travel.
  - **States**: Each cell in the grid is a state.
  - **Actions**: The car can move up, down, left, or right.
  - Rewards:-
    - +10 for reaching the destination.
    - -10 for hitting an obstacle.
    - -1 for each move to encourage the car to find the shortest path

DQN (Implementation)

```python
import numpy as np
import random
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

# Define the environment
GRID_SIZE = 5
START_POSITION = (0, 0)
GOAL_POSITION = (4, 4)
OBSTACLES = [(1, 1), (2, 2), (3, 3)]
```

```
[ (0,0) (0,1) (0,2) (0,3) (0,4) ]
[ (1,0)  O   (1,2) (1,3) (1,4) ]
[ (2,0) (2,1)  O   (2,3) (2,4) ]
[ (3,0) (3,1) (3,2)  O   (3,4) ]
[ (4,0) (4,1) (4,2) (4,3)  G   ]
```

`GOAL_POSITION = (4, 4)`

- **Purpose**: Represents the destination or target location that the agent (self-driving car) needs to reach in the grid.
- **In the Grid**: In a 5x5 grid, this is the bottom-right corner. The car's goal is to navigate to this position while avoiding obstacles.

`OBSTACLES = [(1, 1), (2, 2), (3, 3)]`

- **Purpose**: Represents locations in the grid where obstacles are placed. The car should avoid these positions to prevent collisions or negative rewards.
- **In the Grid**: These are specific cells in the 5x5 grid where obstacles are located:
  - `(1, 1)`: An obstacle in the second row, second column.
  - `(2, 2)`: An obstacle in the third row, third column.
  - `(3, 3)`: An obstacle in the fourth row, fourth column.

- Visual representation of the 5x5 grid with the goal and obstacles.
- G represents the goal position at (4,4). Agent should achive.
- O represents the obstacles. Agent should avoid to maximize the reward.

DQN (Implementation)

1

```python
# Define the Q-Network model
def build_model(state_size, action_size):
    model = Sequential()
    model.add(Dense(24, input_dim=state_size, activation='relu'))
    model.add(Dense(24, activation='relu'))
    model.add(Dense(action_size, activation='linear'))
    model.compile(loss='mse', optimizer=tf.keras.optimizers.Adam(0.001))
    return model
```

2

```python
# Environment setup
def get_reward(state):
    if state == GOAL_POSITION:
        return 10
    elif state in OBSTACLES:
        return -10
    else:
        return -1
```

3

```python
def get_next_state(state, action):
    x, y = state
    if action == 0 and x > 0:  # Move up
        x -= 1
    elif action == 1 and x < GRID_SIZE - 1:  # Move down
        x += 1
    elif action == 2 and y > 0:  # Move left
        y -= 1
    elif action == 3 and y < GRID_SIZE - 1:  # Move right
        y += 1
    return (x, y)
```

Inputs:

- `state`: The current position of the car, represented as `(x, y)`.
- `action`: The action the car will take, represented by an integer (0 to 3).

Discrete Actions: The car can move in four possible directions:

- 0: Move Up
- 1: Move Down
- 2: Move Left
- 3: Move Right

DQN (get_next_state method explanation)

Actions: x, y cordinate managment

- **Action 0**: Move Up
  - **Condition**: `x > 0` ensures that the car does not move above the top boundary of the grid.
  - **Update**: `x -= 1` moves the car one unit up. → adjust x cordinate
- **Action 1**: Move Down
  - **Condition**: `x < GRID_SIZE - 1` ensures that the car does not move below the bottom boundary of the grid.
  - **Update**: `x += 1` moves the car one unit down.
- **Action 2**: Move Left
  - **Condition**: `y > 0` ensures that the car does not move to the left of the grid boundary.
  - **Update**: `y -= 1` moves the car one unit to the left.
- **Action 3**: Move Right
  - **Condition**: `y < GRID_SIZE - 1` ensures that the car does not move beyond the right boundary of the grid.
  - **Update**: `y += 1` moves the car one unit to the right.

DQN (Implementation)

```
def train_dqn(episodes=1000, gamma=0.95, epsilon=1.0, epsilon_min=0.01, epsilon_decay=0.995):
    state_size = GRID_SIZE * GRID_SIZE
    action_size = 4  # Up, Down, Left, Right
    model = build_model(state_size, action_size)
```

Epsilon (ε) : -

- Parameter used to control the level of exploring the environment.
- High values trigger more exploration with random attempts to learn the environment.
- Lower values restrict the random attempts as it could over-learn with too much exploring leading to overfitting.

Epsilon Decay :-

- Reduces the rates set for exploring the environment gradually. Along with the time, learning should be reduced, unless it could lead to overfitting.
- Epsilon starts high to encourage exploration and is reduced over time to encourage exploitation of learned strategies. (to avoid overfitting.)
- For example, If epsilon starts at 1.0 and decays by a factor of 0.995 per episode, the new epsilon value will be epsilon * 0.995 after each episode, reducing the exploration rate gradually.

Gamma (γ)

- **Gamma (γ) :-**
- To determine the gravity of the current vs future rewards. Referred to as the discount factor as well. Ranges between 0 to1.
- **High (close to 1)**: The agent **thinks about the future a lot**. Recommend future moves thinking a lot.. **Presently** the move **could be not be a good** decision. But from a future perspective might be good.
- **Low (close to 0)**: The agent cares mostly about **immediate rewards**. Recommend future moves thinking about the current status. **Presently** the move could be a **good** decision. But from a future perspective might not be good.

Epsilon Min (ε_min)

- This prevents the decay to reach to 0. If so, no exploration will occur.
- This allows the decay to reduce up to the ε_min point and keep statically in that level, without going to 0.
- Then, still though it`s at the lowest level, still slowly explorations will happen.
- This will prevent missing of global opima, while being stuck in local optima.
- In dynamic environments where conditions might change over time, having a small, non-zero exploration rate allows the agent to adapt to new situations and update its policy as needed

Epsilon Min (ε_min)

- Imagine an agent navigating a maze. As the agent learns to navigate the maze effectively, it will start to exploit its learned path (gradually lowering the epsilon).
- However, if ε_min is set to 0.01, the agent will still occasionally try new paths or actions.
- This helps in adapting if the maze changes or if it encounters a new maze configuration.
- Epsilon Min (ε_min) ensures that the agent maintains a minimal level of exploration, which prevents it from totally giving up on it` s exploration policy leading into learning.

DQN (Implementation)

```
for episode in range(episodes):
    state = START_POSITION
    state_vector = np.identity(GRID_SIZE)[state[0]] + np.identity(GRID_SIZE)[state[1]]
    total_reward = 0
```

- `for episode in range(episodes):`: This starts the loop for each training episode. An episode is one complete run through the environment from start to finish. ← iteration
- `state = START_POSITION`: Sets the starting position of the car in the grid. ← (0,0)
- `state_vector = np.identity(GRID_SIZE)[state[0]] + np.identity(GRID_SIZE)[state[1]]`: Converts the state (position) into a one-hot encoded vector. This vector represents the current state in a way the neural network can understand.
- `total_reward = 0`: Initializes the total reward for this episode.

DQN (Implementation)

```
for step in range(100):
    if random.uniform(0, 1) < epsilon:
        action = random.randint(0, action_size - 1)  # Explore
    else:
        action_values = model.predict(state_vector.reshape(1, -1))
        action = np.argmax(action_values[0])  # Exploit
```

- `for step in range(100):`: Runs a loop for a fixed number of steps within each episode (in this case, 100 steps).
- `if random.uniform(0, 1) < epsilon:`: Decides whether to explore or exploit based on the current epsilon value.
  - `random.uniform(0, 1)`: Generates a random number between 0 and 1.
  - `< epsilon`: If this random number is less than epsilon, the agent will explore.
- `action = random.randint(0, action_size - 1)`: **Explore**: Chooses a random action from the available options. This helps the agent try new things and discover potentially better strategies.
- `else:`: If the random number is not less than epsilon, the agent will exploit.
  - `action_values = model.predict(state_vector.reshape(1, -1))`: **Exploit**: Uses the neural network to predict the Q-values (expected rewards) for each action given the current state.
  - `action = np.argmax(action_values[0])`: Chooses the action with the highest predicted Q-value. This means the agent is using its current knowledge to pick the best action.

- **Exploration (Random Action): -**
  - **When random.uniform(0, 1) < epsilon**, the agent picks a random action to explore new possibilities.
- **Exploitation (Best Action)**: :-
  - **When random.uniform(0, 1) >= epsilon**, the agent uses its current knowledge (neural network predictions) to pick the best-known action
- This approach helps the agent learn effectively by balancing exploration (trying new actions) and exploitation (using what it has learned)

DQN (Implementation)

```
# Get the next state after taking the action
next_state = get_next_state(state, action)

# Convert the next state to a one-hot encoded vector
next_state_vector = np.identity(GRID_SIZE)[next_state[0]] + np.identity(GRID_SIZE)[next_state[1]]

# Get the reward for the new state
reward = get_reward(next_state)

# Update the total reward for this episode
total_reward += reward

# Calculate the target value for training
target = reward + gamma * np.amax(model.predict(next_state_vector.reshape(1, -1))[0])
```

Reward Prediction

```
target = reward + gamma * np.amax(model.predict(next_state_vector.reshape(1, -1))[0])
```

- This predicts the Q-values (expected rewards) for all possible actions in the next state.
- np.amax() gets the maximum value from the predicted Q-values. It represents the best possible reward the agent can expect from the next state.
- Imagine the agent is playing a game and the following happens:-
  - When the agent takes an action and moves to the next state, it receives a reward of 10 points.
  - Set to 0.9, meaning the agent values future rewards. Thinks more about the future.
  - Predicted Q values for the completed next state.
    - Action 1: 5 points.
    - Action 2: 7 points.
    - Action 3: 3 points.
  - The highest Q-value is 7 (from Action 2).
- Use the formula: target = reward + gamma * max(Q-values)
- Substituting the values: target = 10 + 0.9 * 7
- Calculation: target = 10 + 6.3 = 16.3.
- The target value 16.3 represents the expected reward value for the action taken.
- This target represents what the model should have predicted for the action taken.

Q Value Updation.

- The current prediction of the model for the action taken is derived.

```
target_vector = model.predict(state_vector.reshape(1, -1))[0]
target_vector[action] = target
```

- This prediction could deviate from the calculated target from previous slide.
- Hence, update the Q-value for the action taken to reflect what is needed as the target value.

```
# Train the model with the updated Q-values
model.fit(state_vector.reshape(1, -1), target_vector.reshape(-1, action_size), epochs=1, verbose=0)
```

Obtain the updated model

```
# Update the state and state vector for the next step
state = next_state
state_vector = next_state_vector

# Check if the episode should end (goal reached or obstacle hit)
if state == GOAL_POSITION or state in OBSTACLES:
    break

# Update epsilon to gradually reduce exploration
epsilon = max(epsilon_min, epsilon * epsilon_decay)

# Print the results of the episode
print(f"Episode {episode + 1}/{episodes}, Total Reward: {total_reward}")

return model;
```

Test the DQN Model

```
# Test the trained model
def test_dqn(model):
    state = START_POSITION
    state_vector = np.identity(GRID_SIZE)[state[0]] + np.identity(GRID_SIZE)[state[1]]
    path = [state]

    for step in range(100):
        action = np.argmax(model.predict(state_vector.reshape(1, -1))[0])
        next_state = get_next_state(state, action)
        path.append(next_state)
        state = next_state
        state_vector = np.identity(GRID_SIZE)[next_state[0]] + np.identity(GRID_SIZE)[next_state[1]]

        if state == GOAL_POSITION or state in OBSTACLES:
            break

    return path
```

```
# Display the path found by the DQN agent
path = test_dqn(model)
print("Path taken by the car:", path)
```